

THOR: Lightning Detection & Alert System

Eoghan Ó Bogail



Stand number: 4201
Pobalscoil Inbhear Scéine
Project Diary

This is the project diary for THOR. It documents the progress I made on the project on a week by week basis. This diary references a lot of terms explained in the **Project Book**, please refer to that for THOR-specific definitions.

Contents

1	September 2024	1
1.1	Week 1	1
1.2	Week 2 and Week 3	1
2	October	2
2.1	Week 4	2
2.2	Week 5	3
2.3	Week 6 and Week 7	4
3	November	5
3.1	Week 8 and Week 9	5
3.2	Week 10 and Week 11	5
4	December	7
4.1	Week 12	7

Chapter 1

September 2024

Note: work on THOR did not begin until 2024-09-11.

1.1 Week 1

Started work on project. Created Git repository for it, committed a boilerplate Flask app and an unattended install script for Debian. The install script installs dependencies like Python, python3-venv, Git, and the Mosquitto MQTT broker, then creates a UNIX service user for THOR, creates configuration folders, sets up a SystemD¹ service, configures Mosquitto, and installs THOR from its Git repository.

1.2 Week 2 and Week 3

Wrote up and submitted my project to BT Young Scientist 2025.

¹Service manager that Debian is currently using.

Chapter 2

October

2.1 Week 4

Updated the install script to start creating configuration files.

Created the first Provider to get the latest forecast from MET Norway's Locationforecast 2.0 API. Started work on mapping MET Norway's icon set to Font Awesome Free in the following function:

```
# /thor/providers.py, commit 474ef88, starts line 28
def met_no_symbol_to_font_awesome(icon):
    """Converts a MET Norway symbol name to a similar Font
    Awesome icon and summary of icon."""
    mapping = {
        "clearsky_day": ("sun", "Sunny"),
        "fair_day": ("cloud-sun", "Fair skies"),
        "cloudy": ("cloud", "Cloudy"),
        "rain": ("cloud-rain", "Raining"),
        "partlycloudy_night": ("cloud-moon", "Partly cloudy")
    }
    print(icon)
    if icon in mapping:
        print(mapping.get(icon))
        return mapping.get(icon, ("circle-question", "Unknown"))
    return "circle-question", "Unknown" # Editors note: Not
    sure what I was doing here.
```

This function has multiple problems, two being that this function will be very hard to maintain and it will be hard to support more than one iconset.

Added Socket.IO and MQTT to the project for connecting to the Dashboard and Nodes respectively. Added support for querying the weather over Socket.IO and MQTT.

Began using SCSS¹ to write the styling for the Dashboard, compiling it using a Python library called webassets

Added Bootstrap² and Font Awesome³ to the project for the Dashboard.

Added a package called Flask-APScheduler, which allows me to send weather updates at a certain interval, like every ten minutes.

2.2 Week 5

Made THOR report that it is online over MQTT on connection, and what the current time is. The hope is that connected nodes can use this for synchronisation.

Started implementing caching in the MET Norway Locationforecast Provider. This is done using a class called CachingWeatherProvider that simply checks if there's a cache and if it expires.

```
# /thor/providers.py, commit a33720c, starts line 45
class CachingWeatherProvider(WeatherProvider):
    def __init__(self, lat: float, long: float):
        super().__init__(lat, long)

        self.last_fetched_time = 0
        self.seconds_to_live = 6000 # 10 minutes
        self.cached_response = {}

    def fetch(self) -> dict:
        if (time.time() > last_fetched_time + seconds_to_live):
            return cached_response
```

However, actually fetching the data is still the job of the specific Provider for MET Norway. This isn't ideal long term because I would have to implement the same code across every Provider that wants to make use of caching.

```
# /thor/providers.py, commit a33720c, starts line 57
# Code has been shortened to save space.
class MetNoWeatherProvider(CachingWeatherProvider):
    def __init__(self, lat: float, long: float):
        # ...

    def fetch(self) -> dict:
        """Fetches data from the MET Norway Location Forecast API"""

        cache = super().fetch()

        if not cache:
            # ... (Making the request)
```

¹Scripting language that is intended to be compiled into CSS

²Web UI framework written in SCSS and JavaScript

³Icon set

```
        self.cache = data
    else:
        data = cache
```

Added a weather widget to the THOR Dashboard. It uses Socket.IO to get the latest weather from the server rather than using Server-Side Rendering.

I also started experimenting with the Celery Task Queue to implement UDP autodiscovery of nodes.

I got the components I need for the Lightning Node and the Display Node. Began working on the display node, showing the weather data from MET Norway on it. It renders the data to an image before sending it to an Inky pHAT using the Pillow Python library. Made it play sound files on command using MPV.⁴ I can't share any of the code because I lost the backup of it.

I also made the realisation that the lightning sensor I got, the DFRobot Gravity Lightning Sensor, wouldn't talk to the Raspberry Pi Pico W I got for it, but it would work with an Arduino Uno (which doesn't have Wi-Fi and wouldn't work in THOR) and a Raspberry Pi 4 (which is overkill for just a sensor like this), so I ordered an Arduino Nano ESP32 in hopes that would work with the sensor.

2.3 Week 6 and Week 7

I forgot to keep a log of my progress over these weeks, so I have to merge them together.

Rewrote the display node code as the code I had written before was hard to expand. Created a class `EPaperImage` to represent an image, and another class `BodyStackItem` for alerts to be stacked on top of each other in the `EPaperImage`. The script checks how many alerts can be shown on screen at once (for the pHAT, it should be a maximum of 2) and then establishes a connection to the MQTT broker on the Hub. At the moment, the username, password and broker ip address are hardcoded, which makes it annoying to change.

When a message is received, either a weather or an alert message, the script creates an image using the Pillow library and draws the information onto the image, before sending it to the display.

On the Hub, I added a database to store alerts in that uses SQLite, and created a class called `Alert` to represent an alert. I then derived from that another alert `LightningAlert` to quickly configure parameters on `Alert`. I also created a new Provider to fetch weather warnings from Met Éireann, and a new alert type to go with it. When the Hub is fetching alerts from the database, it checks if there's any weather warnings and includes them in the response if there are.

⁴Cross-platform media player like VLC

Chapter 3

November

3.1 Week 8 and Week 9

On Week 8 and 9 I did not work on the project due to my mid-term break and other commitments in school.

3.2 Week 10 and Week 11

I wrote an autodiscovery script that works by listening to UDP packets from nodes, based off code I found on GitHub by user jholtmann, and added a new SystemD service for it. I also began work on an API to allow clients to set and retrieve settings stored in the database, as well as define an endpoint for nodes to self-register at. I also started looking into how to get around FlaskMQTT, the package I was using for MQTT support on the Hub, only supporting running on one process at a time (if I ran more than one instance of the hub for performance reasons the response over MQTT would be sent more than once). At this time, I simply disabled it in the Flask app factory config unless it was being enabled explicitly when it was run. I also started working on a universal installer for THOR, but I chose not to complete it to focus on other parts of the project.

I also started work on the Alerts program for my desktop. I was originally going to design it using the Qt GUI framework, but I ended up choosing wxFrames (which I would be accessing using the wxPython module) as it would be easier to create a system tray icon. The program connects to the Hub using Socket.IO rather than MQTT to avoid needing to do UDP autodiscovery and registration (this has the benefit of allowing the user to pick a Hub that isn't on the local network). At the moment, it only shows the current weather.

I had to create an icon set for this, called THORICONS, because not every icon set has all the icons I want and icons aren't consistent across different

platforms anyway¹. To avoid having to draw an icon for a specific scenario, I designed a system where a script would draw different images on top of each other to create a single icon.

```

from PIL import Image
# /icongenerator/generate.py starting line 19, commit b955cd
for icon in needed_icons:
    new_icon = Image.new("RGBA", (32, 32))

    # Split icon names into their different parts and load the
    # images
    parts = icon.split("-")
    # Set variations of parts
    if 'rain' in parts:
        if {'rain', 'lightning'}.intersection(set(parts)):
            parts[parts.index('cloud')] = 'cloud_grey' # Grey clouds
            if raining
    if len(parts) <= 1:
        if 'sun' in parts: # Show a large sun if no other parts in
            image
            parts = ['sun_large']
        if 'moon' in parts:
            parts = ['moon_large']

    # Try load the icons
    for part in parts:
        if part not in image_parts:
            try:
                image_parts[part] = Image.open(os.path.join('img',
                    f"{part}.png"))
            except FileNotFoundError:
                image_parts[part] = Image.new("RGBA", (32, 32))

        # Overlay the part onto the image
        new_icon.paste(image_parts[part], mask=image_parts[part])

    new_icon.save('../assets/icons/' + icon + '.png', 'PNG')

```

I drew the icons in a pixel art style using the software Aseprite, because I'm familiar with drawing pixel art and it is easy to do.

¹Or even across the same platform. Windows 8's iconography is completely different from Windows 11's.

Chapter 4

December

4.1 Week 12

I began implementing UDP autodiscovery on the nodes, as before it was hard-coded. To do this, I created a simple way to read and write to a JSON file serving as a configuration file to save the server settings to avoid searching for a Hub every run. Then I broadcast a UDP packet to 255.255.255.255¹ and hit the node registration endpoint on the Hub that responds to get MQTT credentials. I added this to the display node and I began writing the lightning provider node. This is when I found out that the lightning sensor wouldn't talk to the Arduino Nano either, so I made the Arduino Uno send the data from the sensor to the Nano over a UART serial connection, which would then send it to the Hub over MQTT.

On the Hub, I moved MQTT into its own Python module because having the Alerts app open was causing the Hub to freeze, so I needed to run more than one instance of it. The MQTT module (or bridge, I guess) sends MQTT messages to the Hub over the Hub's Socket.IO server, in a different 'namespace' to allow the Hub to know if the message is from MQTT. I also made the node registration API return the MQTT broker details, username and password.

I also refactored the THOR Alerts app into a Python module to clean up the code base a little bit.

4.2 Week 13

I focused on creating my project's video this week.

I also added proper support for notifications and receiving alerts in the Alerts app, so it now sends you notifications.

¹Which means every device on the network

4.3 Week 14 and 15

I mainly worked on my poster this week.

I added alerts to the dashboard, using Socket.IO and JavaScript to dynamically fetch and display them rather than using server-side rendering, which allows the dashboard to update in real time. I also made the database table for alerts more generic to properly support all types of alerts. I also added a settings page to THOR, allowing you to set the location for the weather and the county for weather warnings, and turn external providers on and off. This allowed me to stop hardcoding locations and providers.

I made the Alerts app show a warning icon if there is an active alert.

I fixed a bug in the display node where content that was very long (like when there's several weather warnings) wouldn't get detected as JSON content and wouldn't get parsed.

My project is now in a state where everything is working.

I finished my project book.